

青凤

先祖父在时，曾于后园植柏一株。那柏树年深日久，苍翠蓊郁，枝干盘曲如龙。我幼时常于月夜见一女子立于树下，衣青如叶，面白如月。问其名，不答，但笑而已。后数年，不复见。

及冠，偶得一册《青凤斋笔记》，纸页焦黄，字迹娟秀。开卷便是两行小字：

@ 外景

1

```
#include <stdio.h>
#include <stdlib.h>
=> 重现庭中气象 <9>
```

我不解其意，却觉这符号里藏着某种秩序，仿佛一扇门的榫卯。笔记接着说，青凤斋非寻常四合院，其制奇特：中央一亭，左右各有长廊，廊外各有小院，院中又有夹道。来客不论多少，入此庭中，皆不觉拥挤，亦不觉空寂。

笔记里画了一幅草图，并附有一段奇怪的记号：

@ 庭院节点

2

```
typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
    int height;
} Node;
=> 重现庭中气象 <9>
```

那图中的亭台廊道，竟与这记号一一对应。我想，这大约便是青凤斋的营造法式，只是以我未曾见过的文字写成。

青凤每日子时踏月，循廊而舞。笔记中录有她的步法根基：

@ 求高与取大

3

```
int height(Node *n) {
    if (n == NULL) return 0;
    return n->height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}
=> 重现庭中气象 <9>
```

这两个动作极是简单，不过是探知脚下高低，比较轻重。但青凤将它们融在舞步里，便生出无穷变化。

我最感兴趣的，是她的“左揽”与“右牵”。据笔记载，若庭院西北角偏重，青凤便以右足为轴，轻轻一引，将重负移至东南。那姿态舒展如揽月入怀。

@ 左揽

4

```
Node *rotate_right(Node *y) {
    Node *x = y->left;
    Node *T2 = x->right;
```

```

x->right = y;
y->left = T2;

y->height = max(height(y->left), height(y->right)) + 1;
x->height = max(height(x->left), height(x->right)) + 1;

return x;
}

```

=> 重现庭中气象 <9>

若东南过沉，她便左足一旋，往右带入，仿佛牵动一片流云。

@ 右牵 #

5

```

Node *rotate_left(Node *x) {
    Node *y = x->right;
    Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

```

=> 重现庭中气象 <9>

我曾于月下窥见一回。她舞时衣袖交叠，环佩无声，只觉四周的空气微微震荡，廊柱间的影子轻轻移动。舞罢归亭，庭院里万籁俱寂，仿佛一切重物都找到了最妥帖的位置。

笔记中还有“安宅”之法。每有新客至，青凤便引他入庭，视其轻重，安置于某院某角。置毕，必从头踏舞一遍，使四方重新匀停。这一套动作记录得分外详密：

@ 安宅 #

6

```

Node *insert(Node *node, int key) {
    if (node == NULL) {
        Node *new_node = (Node *)malloc(sizeof(Node));
        new_node->key = key;
        new_node->left = NULL;
        new_node->right = NULL;
        new_node->height = 1;
        return new_node;
    }

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;
}

```

```

node->height = 1 + max(height(node->left), height(node->right));

int balance = height(node->left) - height(node->right);

if (balance > 1 && key < node->left->key)
    return rotate_right(node);

if (balance < -1 && key > node->right->key)
    return rotate_left(node);

if (balance > 1 && key > node->left->key) {
    node->left = rotate_left(node->left);
    return rotate_right(node);
}

if (balance < -1 && key < node->right->key) {
    node->right = rotate_right(node->right);
    return rotate_left(node);
}

return node;
}
=> 重现庭中气象 <9>

```

我反复揣摩这段记号，渐渐明白：那“左揽”“右牵”并非孤立舞姿，它们与“安宅”连为一体，恰如一首曲子里的叠句与回旋。

某岁重九，青凤忽邀我遍览诸院。我随她穿廊过角，所见次序井然：先入左首小院，再及夹道，最后至右首长廊尽头。我暗记路径，归来后竟能写下这样一段游廊次序：

④ 游廊

7

```

void inorder(Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}
=> 重现庭中气象 <9>

```

这大约便是她庭院的全部奥秘：无论来客如何乱序而入，游观之时，总能依着某种无形的规矩，从小到大，一一经过。

笔记末页，青凤写道：“余将远行，此斋留与有缘。但取《守衡》一节研习，自能复现庭中气象。”接着便是一段离去的仪式：

④ 归去

8

```

void free_tree(Node *root) {
    if (root != NULL) {
        free_tree(root->left);
        free_tree(root->right);
        free(root);
    }
}

```

```
}  
}  
=> 重现庭中气象 <9>
```

我终于下定决心，将笔记中所有记号按序誊录在一处。那些零散的符号，那些左揽右牵、安宅游廊，竟能拼合成一个完整的仪式：

⑨ 重现庭中气象 #

9

```
# 外景 ⑨ <1>  
# 庭院节点 ⑨ <2>  
# 求高与取大 ⑨ <3>  
# 左揽 ⑨ <4>  
# 右牵 ⑨ <5>  
# 安宅 ⑨ <6>  
# 游廊 ⑨ <7>  
# 归去 ⑨ <8>
```

```
int main() {  
    Node *root = NULL;  
    int keys[] = {30, 20, 40, 10, 25, 35};  
    int n = sizeof(keys) / sizeof(keys[0]);  
  
    for (int i = 0; i < n; i++) {  
        root = insert(root, keys[i]);  
    }  
  
    printf("青凤斋游观次序: ");  
    inorder(root);  
    printf("\n");  
  
    free_tree(root);  
    return 0;  
}
```

我将这一段仪轨录于纸上，置于案头。窗外柏树萧萧，月影西斜。我不知道青凤是否还会再来。但每当我运行这套仪轨，那庭院便会在纸上重新立起，廊柱井然，宾客咸集，而庭基永不偏斜。

也许青凤从未离去。她只是将自己的舞步，藏进了这些奇怪的符号里，等一个有缘人，将它们重新唤醒。